

- Lab Title: Secure Coding Practices and Input Validation
- Assignment Title: Hands-on Secure Coding & Code Review
- Student Name: Bashir Adam
- Student ID: 2025/FWSD/11509
- Course Name: BVWS102 – Web Application Security Essentials
- Instructor Name: Aminu Idris
- Date of Submission: 29 Apr, 2026
- Version: Version 6.1

LAB OVERVIEW/OBJECTIVE

This comprehensive lab is meticulously designed to provide an in-depth, hands-on experience with the principles of secure coding as presented in the BVWS106 course. Covering over 100 slides of material, this assignment will rigorously test your theoretical understanding, your ability to identify Common Weakness Enumerations (CWEs), and your practical skills in detecting and remediating vulnerabilities across various security mechanisms in both PHP and Python applications.

Part 1: Core Principles and Foundational Concepts

1.1 The CIA Triad

Confidentiality: Ensures information is only accessible to authorized users.

Integrity: Ensures data is correct and has not been altered.

Availability: Ensures information is accessible when needed by legitimate users.

Attack	CIA Impact
SQL Injection	Confidentiality (data exposure), Integrity (data modification)
Denial of Service (DoS)	Availability
Unauthorized Data Access	Confidentiality

1.2 Security Principles in Practice

1. Minimize Attack Surface

Reduce points where attackers can interact with the system.

Example: Only allow required input fields instead of multiple unnecessary ones.

2. Establish Secure Defaults

System should be secure without requiring manual configuration.

Example: Ports closed by default unless explicitly opened.

3. Least Privilege

Users should only have access they absolutely need.

Example: Regular users cannot access admin features.

4. Defense in Depth

Use multiple layers of security instead of relying on one.

Example: Login + input validation + logging.

5. Fail Securely

System should default to a secure state when failure occurs.

Example: Deny access if authentication check fails.

1.3 Common Threats

1. Cross-Site Scripting (XSS)

XSS occurs when an attacker injects malicious scripts (usually JavaScript) into a trusted website. When users visit the affected page, the script runs in their browser, allowing the attacker to steal sensitive data (like cookies or login details), hijack sessions, or manipulate the webpage content.

2. Cross-Site Request Forgery (CSRF)

CSRF tricks a logged-in user into performing unwanted actions on a web application without their consent. The attacker sends a malicious request (e.g., via a link or hidden form), and because the user is already authenticated, the system processes it as a legitimate action.

3. Broken Authentication and Session Management

This happens when authentication systems are poorly implemented, allowing attackers to compromise passwords, session tokens, or user identities. Examples include weak passwords, exposed session IDs, or failure to properly log out users, leading to account takeover.

4. Insecure Deserialization

Insecure deserialization occurs when untrusted data is used to reconstruct objects without proper validation. Attackers can manipulate serialized data to execute malicious code, escalate privileges, or perform attacks like remote code execution.

Part 2: CWE Identification and Mitigation Strategies

Vulnerability Description	CWE ID (e.g., CWE-89)	Mitigation Strategy
The software constructs all or part of an SQL command using externally-influenced input from an upstream component, but it does not neutralize or incorrectly neutralize special elements that could modify the intended SQL command.	CWE-89	Validate input, avoid concatenation
The product does not require users to have strong passwords, which makes it easier for attackers to compromise user accounts.	CWE-521	Enforce strong password rules
The software does not implement sufficient measures to prevent multiple failed authentication attempts within in a short time frame, making it more susceptible to brute force attacks.	CWE-307	Implement account lockout
The web application uses the GET method to process requests that contain sensitive information, which can expose that information through the browser's history, referers, web logs, and other sources.	CWE-598	Use POST method
The lack of proper session expiration may improve the likely success of certain attacks.	CWE-613	Implement session expiration
Authenticating a user, or otherwise establishing a new user session, without invalidating any existing session identifier allows an attacker to steal authenticated sessions.	CWE-384	Regenerate session ID
The software does not neutralize or incorrectly neutralizes user-controllable input before it is placed in output that is	CWE-79	Encode output

used as web page content, leading to the execution of malicious client-side scripts.		
The software does not sufficiently verify whether a request was intentionally created by the user, allowing an attacker to trick the user into performing actions they did not intend. The software does not properly restrict access to resources based on the user's privileges, allowing unauthorized users to perform actions or access data they should not.	CWE-425	Verify request authenticity & Enforce access control
The software does not properly handle errors, leading to the disclosure of sensitive information or a denial of service.	CWE-200	Avoid exposing sensitive errors

Part 3: Authentication and Password Managemen

3.1 Theory Questions

1. Authentication vs Authorization

- ☐ Authentication: Verifies identity
- ☐ Authorization: Determines what user can do

Why separate?

Because verifying identity is different from granting permissions.

2. Three Factors of Authentication

- ☐ Something you know: Password
- ☐ Something you have: Token / SecureID
- ☐ Something you are: Fingerprint

3. Weak Password Recovery Risks

- ☐ Allows attackers to reset passwords easily
- ☐ Often weaker than login security

Secure Reset

- ☐ Use strong questions
- ☐ Send temporary password to email
- ☐ Force password change
- ☐ Rate limit requests

3.2 PHP Code Review: Vulnerable Authentication

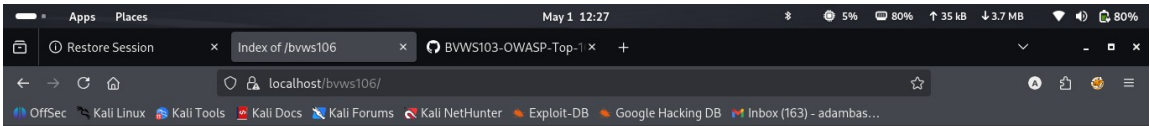
```
GNU nano 8.7.1
azad@Adam:/opt/lampp/htdocs/bvws106
vulnerable.php

// login.php
include("db_config.php");

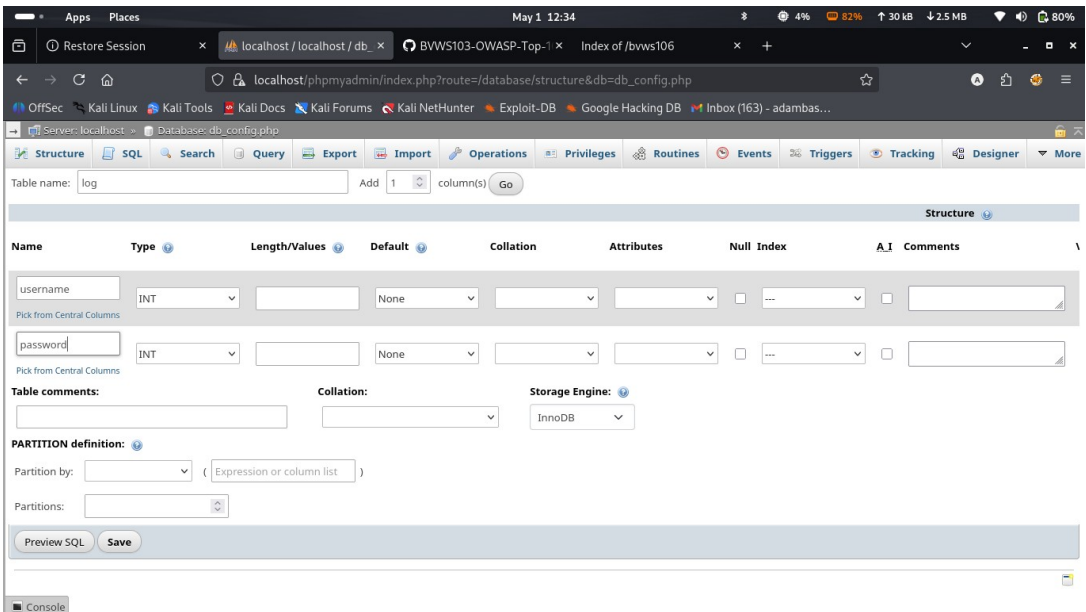
$username = $_REQUEST["username"]; // Using $_REQUEST for flexibility
$password = $_REQUEST["password"];

// Query to check username and password
$sql = "SELECT * FROM users WHERE username = ".$username." AND password = ".md5($password);
$result = $conn->query($sql);

if ($result->num_rows > 0) {
    $row = $result->fetch_assoc();
    session_start();
    $_SESSION["loggedin"] = true;
    $_SESSION["id"] = $row["id"];
    $_SESSION["username"] = $row["username"];
    $_SESSION["role"] = $row["role"];
    header("Location: welcome.php");
}
```



Name	Last modified	Size	Description
Parent Directory	-	-	-
vulnerable.php	2026-05-01 12:22	657	



Vulnerabilities

1. SQL Injection: CWE-89
2. Weak hashing (MD5): susceptible to rainbow tables
3. Uses \$_REQUEST: unsafe input
4. No account lockout: CWE-307

Exploitation Example

Input:

username = 1 OR 1=1

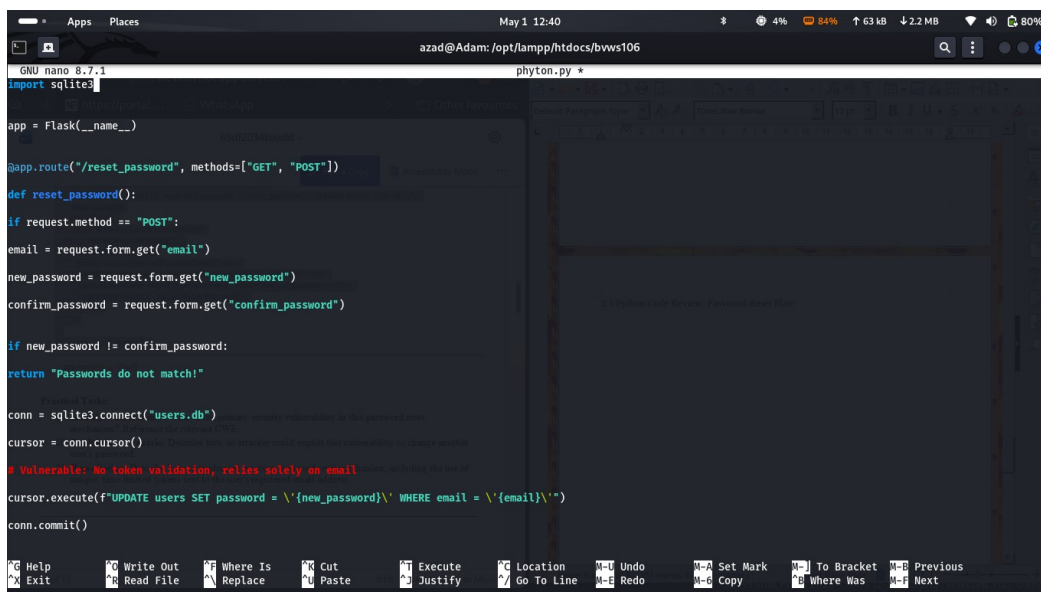
password = anything

This bypasses login.

Remediation

- ☐ Use prepared statements
- ☐ Replace MD5 with stronger hashing
- ☐ Validate inputs
- ☐ Implement account lockout
- ☐ Use POST instead of REQUEST

3.3 Python Code Review: Password Reset Flaw



```
GNU nano 8.7.1 phyton.py
import sqlite3

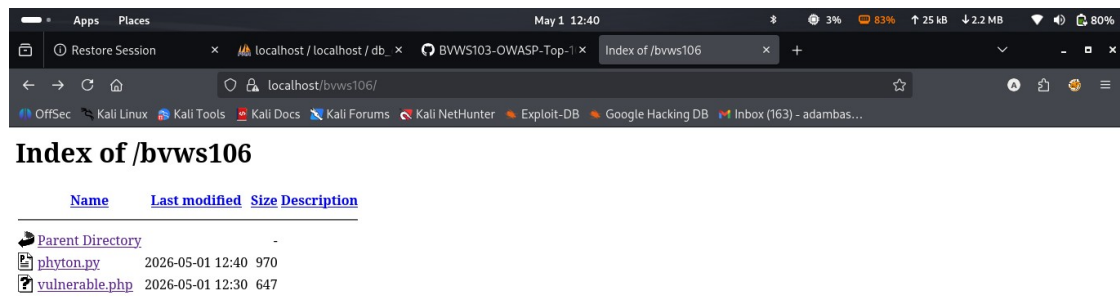
app = Flask(__name__)

@app.route("/reset_password", methods=["GET", "POST"])
def reset_password():
    if request.method == "POST":
        email = request.form.get("email")
        new_password = request.form.get("new_password")
        confirm_password = request.form.get("confirm_password")

        if new_password != confirm_password:
            return "Passwords do not match!"

        conn = sqlite3.connect("users.db")
        cursor = conn.cursor()

        # Vulnerable: No token validation, relies solely on email
        cursor.execute(f"UPDATE users SET password = '{new_password}' WHERE email = '{email}'")
        conn.commit()
```



Index of /bvws106

Name	Last modified	Size	Description
Parent Directory	-	-	-
phyton.py	2026-05-01 12:40	970	
vulnerable.php	2026-05-01 12:30	647	

Vulnerability

- ☐ Weak password reset: CWE-640

Exploitation

Attacker submits victim's email resets password directly.

Fix

- ☐ Use unique token sent to email
- ☐ Token must expire
- ☐ Verify ownership before reset

Part 4: Authorization and Access Control

4.1 Theory Questions

Least Privilege

Users only get minimal access required.

RBAC vs ABAC

(Not explicitly covered keep minimal)

- ☐ RBAC: based on roles
- ☐ ABAC: based on attributes

Privilege Escalation

- ☐ Horizontal: accessing another user's data
- ☐ Vertical: gaining admin privileges

4.2 PHP Code Review: Insecure Direct Object Reference (IDOR)

Vulnerability

- ☐ No authorization check

Exploit

Change URL: ?id=2

Access another user's data.

Fix

Check: if (\$user_id != \$_SESSION["user_id"]) deny access

4.3 Python Code Review: Insecure Authorization Logic

Weakness

- ☐ Trusts session blindly
- ☐ Violates “Don’t Trust Services” principle

Exploit

Attacker manipulates session: sets role = admin

Fix

- ☐ Validate role from database
- ☐ Do not trust session alone

Part 5: Session Management

5.1 Theory Questions

Session Fixation

Attacker sets session ID before login.

Session Expiration

Without it: attackers reuse sessions.

Secure Session ID

- ☐ Random
- ☐ Unique
- ☐ Hard to guess

5.2 PHP Code Review: Session Fixation Vulnerability

Identify Vulnerability

The vulnerability present is Session Fixation.

- ☐ CWE Reference: CWE-384: Session Fixation

Exploitation Scenario

An attacker can first create or obtain a valid session ID from the application. They then trick a victim into using that same session ID (e.g., by sending a crafted link).

Remediation (Secure Code)

Regenerate session after login: `session_regenerate_id(true);`

5.3 Python Code Review: Insecure Session Handling

Identify Vulnerability

Vulnerability:

Use of a hardcoded and weak secret key: Insecure Cryptographic Key Management

```
app.secret_key = "insecure_secret_key_for_demo"
```

- ☐ CWE Reference: CWE-321: Use of Hard-coded Cryptographic Key

Implications:

Flask uses `secret_key` to sign session cookies. If it is weak or exposed:

- ☐ An attacker can forge session cookies
- ☐ They can modify session data (e.g., set `username = "admin"`)
- ☐ They can impersonate any user
- ☐ Leads to full session hijacking / privilege escalation

Remediation

In production, the `secret_key` must be:

* Strong

- ☐ Random and unpredictable

* Securely Generated

Example: `import os`

```
app.secret_key = os.urandom(32)
```

* Properly Managed

- ☐ Store in environment variables, not in code:

```
import os app.secret_key = os.environ.get("SECRET_KEY")
```

Additional Vulnerability

Vulnerability:

Improper Authentication / Trusting User Input

CWE-807: Reliance on Untrusted Inputs in a Security Decision

User-controlled input is directly used to set session data: `session["username"] = request.args["username"]`

Anyone can set any username without authentication

- ☐ Example attack: `/set_session?username=admin`

Part 6: Data Validation and Input Sanitization

6.1 Theory Questions

Never Trust User Input

All user input can be manipulated or malicious.

Validating it prevents attacks like SQL injection and XSS.

Whitelisting vs Blacklisting

- ☐ Whitelisting: Allow only safe inputs
- ☐ Blacklisting: Block known bad inputs

Preferred: Whitelisting more secure and harder to bypass.

Output Encoding

Converts data into a safe format before display to prevent execution.

Example: `<script>` → `<script>`

Used in: Web pages when displaying user input (prevents XSS).

6.2 PHP Code Review: SQL Injection and XSS

Identify Vulnerabilities

- ☐ SQL Injection
 - ☐ CWE-89: Improper Neutralization of Special Elements in SQL Command
 - ☐ Cause: Directly inserting user input into SQL query
- ☐ Cross-Site Scripting (XSS)
 - ☐ CWE-79: Improper Neutralization of Input During Web Page Generation
 - ☐ Cause: Outputting user input without encoding

Exploitation Scenarios

SQL Injection

Attacker submits:

```
username=1, 'attacker' --  
message=anything
```

Query becomes manipulated, allowing:

- ☐ Data tampering
- ☐ Database dumping
- ☐ Bypassing logic

XSS Attack

Attacker submits: `<script>alert('Hacked')</script>`

When displayed:

- ☐ Script runs in users' browsers
- ☐ Can steal cookies or sessions

Remediation (Secure Code)

Use prepared statements:

```
for injection  $stmt = $conn->prepare("INSERT INTO guestbook (username, message)
                VALUES (?, ?)");
                $stmt->bind_param("ss", $username, $message);
                $stmt->execute();
```

```
for XSS        echo "<p><b>" . htmlspecialchars($row["username"]) . " :</b> " .
                htmlspecialchars($row["message"]) . "</p>";
```

Encode output

6.3 Python Code Review: Command Injection

Identify Vulnerability

The primary vulnerability is OS Command Injection due to unsafe concatenation of user input into a shell command.

- ☐ CWE Reference: CWE-78: Improper Neutralization of Special Elements used in an OS Command (OS Command Injection)

Exploitation Scenario

An attacker can inject additional shell commands by manipulating the host input.

Example attack inputs:

- ☐ 8.8.8.8; whoami
- ☐ 8.8.8.8 && id
- ☐ 127.0.0.1 | cat /etc/passwd

Remediation (Secure Version)

Fix the vulnerability by:

- ☐ Removing shell=True
- ☐ Passing command arguments as a list
- ☐ Optionally validating input (recommended)

Part 7: Error Handling and Logging

Why verbose error messages are dangerous

Displaying detailed error messages to end-users is dangerous because it can unintentionally expose sensitive internal system information that attackers can exploit.

What can be leaked:

- ☐ File paths (e.g., /var/www/html/app/config.py)
- ☐ Database structure (table names, queries, schema)
- ☐ Server technology stack (Flask version, Python version, OS type)
- ☐ Stack traces showing internal logic
- ☐ API keys, secrets, or environment variables (in poorly configured systems)
- ☐ Internal IP addresses or network architecture details

Why this matters:

Attackers use this information for reconnaissance, making it easier to:

- ☐ Identify vulnerable components
- ☐ Plan targeted attacks

*Log what matters, but not too much” principle

This principle means logging should be useful for monitoring and investigation, but must avoid creating security risks or excessive noise.

What SHOULD be logged:

- ☐ Authentication events (login success/failure)
- ☐ Privilege changes (e.g., admin access granted)
- ☐ System errors (with safe, non-sensitive details)

What SHOULD NOT be logged:

- ☐ Passwords or password hashes
- ☐ API keys, tokens, or secrets
- ☐ Full personal data (e.g., credit card numbers, full addresses)

Importance of secure logging in incident response and forensics

Secure logging is critical because logs act as the primary evidence source during security investigations.

Key importance:

1. Incident detection

- ☐ Helps identify unusual or malicious behavior early (e.g., repeated failed logins, privilege escalation attempts)

2. Forensic investigation

- ☐ Provides a timeline of attacker actions

- ☐ Helps reconstruct how a breach happened

7.2 PHP Code Review: Insecure Error Handling

Identify Vulnerabilities

There are several security issues in the PHP code, especially around error handling and configuration management:

1. Verbose error disclosure (Insecure Error Handling)

CWE reference: CWE-209: Generation of Error Message Containing Sensitive Information

2. Hardcoded credentials (Bad configuration practice)

CWE reference: CWE-798: Use of Hard-coded Credentials

Remediation strategy:

- ☐ Remove hardcoded credentials → use environment variables
- ☐ Hide error details from users
- ☐ Log detailed errors securely for administrators
- ☐ Use a generic message for users

7.3 Python Code Review: Insufficient Logging

Identify Vulnerability

The main issue is insufficient and insecure logging, especially for a high-risk financial operation (payment processing).

1. Lack of proper security logging

- ☐ Only successful transactions are partially logged
- ☐ Failed transactions are not logged with enough detail

Use of print() instead of proper logging:

```
print(f"Payment processed for user {user_id}, amount {amount}")
```

- ☐ print() is not secure or structured logging
- ☐ Cannot be centrally managed or protected

Relevant CWE references:

- ☐ CWE-778: Insufficient Logging
- ☐ CWE-532: Insertion of Sensitive Information into Log File

Secure Remediation (Improved Logging)

Goals:

- ☐ Use proper logging system
- ☐ Log both success and failure events
- ☐ Avoid logging sensitive raw data

Part 8: Encryption and Data Protection

8.1 Theory Questions

*Encryption at rest vs encryption in transit

Encryption at rest: Data should be encrypted when it is stored and not actively being used or transmitted.

When it is needed:

- ☐ Databases
- ☐ File systems
- ☐ Backups
- ☐ Cloud storage

Examples:

- ☐ Encrypting a customer database stored on a server
- ☐ Full disk encryption on a laptop (e.g., BitLocker or LUKS)
- ☐ Encrypting backup files stored in AWS S3 or Google Cloud Storage

Encryption in transit: Data should be encrypted when it is moving between systems or networks.

When it is needed:

- ☐ Client: server communication
- ☐ API requests
- ☐ File transfers over networks

Examples:

- ☐ HTTPS (TLS) when accessing a website
- ☐ Secure API communication using TLS
- ☐ SSH for remote server login

*Symmetric vs Asymmetric Encryption

Symmetric encryption: Uses one shared key for both encryption and decryption eg: AES (Advanced Encryption Standard)

Use cases:

- ☐ Encrypting databases
- ☐ File encryption
- ☐ VPN data tunnels

Asymmetric encryption: Uses two keys:

- ☐ Public key (encrypt)
- ☐ Private key (decrypt)

Examples:

- ☐ RSA
- ☐ ECC (Elliptic Curve Cryptography)

Use cases:

- ☐ Secure key exchange (e.g., TLS handshake)
- ☐ Digital signatures
- ☐ Email encryption (PGP)

When to use each:

- ☐ Symmetric encryption: for speed and bulk data protection
- ☐ Asymmetric encryption: for secure communication setup, authentication, and key exchange
- ☐ In practice, systems like HTTPS use both together (hybrid encryption)

* Risks of weak or outdated cryptographic algorithms

Using weak or obsolete cryptography exposes systems to serious security risks:

1. Easy decryption by attackers

- ☐ Weak algorithms can be broken using modern computing power
- ☐ Example: DES (obsolete) can be brute-forced quickly

2. Exposure to known vulnerabilities

- ☐ Older algorithms often have published attack methods
- ☐ Example: MD5 and SHA-1 are vulnerable to collision attacks

3. Data breaches and data recovery attacks

- ☐ Encrypted data can be decrypted if algorithm is broken
- ☐ Attackers can recover passwords, financial data, or sensitive files

4. Compliance violations

- ☐ Violates security standards like:
 - ☐ PCI-DSS
 - ☐ ISO 27001
 - ☐ GDPR requirements for data protection

5. Loss of trust and integrity

- ☐ Digital signatures can be forged
- ☐ Data authenticity becomes unreliable

Examples of weak/outdated algorithms:

- ☐ MD5
- ☐ SHA-1
- ☐ DES

Safer modern alternatives:

- ☐ AES-256 (symmetric)
- ☐ SHA-256 / SHA-3 (hashing)

- ☐ RSA-2048+ or ECC (asymmetric)

8.2 PHP Code Review: Insecure Data Storage

Identify Vulnerability

The major security issue is improper storage of sensitive data in plaintext without protection.

- ☐ Sensitive data is written directly to a file (sensitive_data.txt)
- ☐ No encryption, access control, or validation is applied

Relevant CWE: CWE-922: Insecure Storage of Sensitive Information

Remediation

To securely store sensitive data, you should apply encryption at rest + secure storage practices.

1. Encrypt data before storing

Use strong encryption like AES (symmetric encryption).

2. Key security improvements

Encryption at rest

- ☐ Data is encrypted using AES-256
- ☐ Stored data is unreadable without the key

8.3 Python Code Review: Weak Encryption Implementation

Identify Vulnerability

This code implements a custom XOR-based encryption scheme, which contains several serious cryptographic weaknesses.

Major security flaws

1. Weak encryption (XOR with repeating key):

```
encrypted_data += chr(ord(data[i]) ^ ord(key[i % len(key)]))
```

2. Predictable and short key:

```
encryption_key = "weakkey"
```

3. No cryptographic protections

4. Custom cryptography is unsafe

Why custom cryptography is discouraged

- ☐ Cryptography is extremely complex and easy to get wrong
- ☐ Even small design mistakes break security

- ☐ Standard algorithms are:
 - ☐ Tested by experts
 - ☐ Widely analyzed
 - ☐ Resistant to known attacks

Secure Remediation (Using Proper Cryptography in Python)

Use a trusted library like cryptography which implements modern secure algorithms (AES-GCM recommended).

```
AESGCM.generate_key(bit_length=256)
```

Part 9: Secure Code Review Checklist (Advanced)

Secure Code Review Checklist (10 Items)

1. Input Validation Implemented
Ensure all user inputs are validated using whitelisting where possible.
2. Output Encoding Applied
All user-generated content is properly encoded before being displayed to prevent XSS.
3. Prepared Statements Used
All database queries use prepared statements or parameterized queries (no direct SQL concatenation).
4. Authentication Secured
Strong password handling and proper login mechanisms are implemented (no weak or hardcoded credentials).
5. Session Security Enforced
Sessions are regenerated after login and securely managed (CWE-384 prevention).
6. Authorization Checks Present
Users can only access resources they are permitted to view (prevents IDOR - CWE-639).
7. Secret Keys Protected
No hardcoded secret keys; sensitive values are stored securely using environment variables.
8. Protection Against CSRF
State-changing requests include CSRF tokens or equivalent protections.
9. Safe Deserialization Practices
No unsafe deserialization of untrusted data (prevents remote code execution risks).
10. Error Messages Sanitized
System errors do not expose sensitive information (no database structure or system details leaked).

References

- [1] [OpenSecurityTraining.info: IntroSecureCoding.html](https://www.opensecuritytraining.info/IntroSecureCoding.html) - Original course material
- [2] CWE - Common Weakness Enumeration: cwe.mitre.org - For CWE definitions and details
- [3] OWASP Top 10: owasp.org/www-project-top-ten/ - For common web application security risk